

Assignment 3

Objective

The goal of this assignment is to:

- implement **binary logistic regression** in PyTorch using only tensors,
 - compute gradients for logistic regression **manually** (without using `.grad` for parameter updates),
 - implement **gradient descent manually**,
 - build and train an **MLP with 1 or 2 hidden layers**,
 - understand binary classification with **sigmoid + binary cross-entropy**,
 - work with an image dataset by **flattening images into vectors**.
-

General Requirements

- Use **PyTorch**
- Submit a **Jupyter Notebook (.ipynb)**
- Notebook must:
 - run from top to bottom without errors,
 - include markdown explanations,
 - include plots where required,
 - set a random seed,
 - use clear variable names,
 - be reproducible.

You may use a dataset such as **Fashion-MNIST**

 docs.pytorch.org/vision/main/generated/torchvision.datasets.FashionMNIST.html

Restrictions (Important)

For the entire assignment:

 Allowed:

- `torch.Tensor` operations
- `torch.exp`, `torch.log`, `torch.matmul`, `@`
- indexing, masking, reshaping, flattening
- `torchvision.datasets` only for loading data
- plotting libraries such as `matplotlib`

✗ Not allowed in **Task 1**:

- `loss.backward()`
- `.grad` for computing logistic regression parameter updates
- `torch.optim.*`
- `torch.nn.Linear`
- high-level loss modules such as `nn.BCELoss`, `nn.BCEWithLogitsLoss`
- high-level training wrappers

✗ Not allowed in **all tasks**:

- `torch.optim.*`
- high-level training wrappers (Lightning, fastai, Trainer APIs)

⚠ For the MLP tasks, you may use autograd via `loss.backward()`, but you must still implement:

- the forward pass yourself,
- the parameter tensors yourself,
- the parameter updates manually.

Dataset Construction for All Tasks

Use an image dataset such as **Fashion-MNIST**.

Each image should be treated as a vector:

$$x \in \mathbb{R}^{784}$$

by flattening the 28×28 image into a 1D vector.

Binary Classification Setup

Choose **one class as the positive class** and treat all other classes as negative.

For example:

- positive class: **Pullover**,
- negative class: all other Fashion-MNIST classes.

To make the dataset balanced:

1. keep **all positive examples** of the chosen class,
2. randomly sample the **same number of negative examples** from the remaining classes,
3. combine them into one binary dataset.

Thus the labels are:

$$y = \begin{cases} 1 & \text{positive class} \\ 0 & \text{negative class} \end{cases}$$

Requirements

- Create a balanced binary dataset.
 - Split into **train / validation** sets.
 - Use the **same split** wherever comparison is required.
 - Report:
 - which class you chose as positive,
 - the number of positive and negative samples,
 - the shapes of the resulting tensors.
-

Task 1 — Logistic Regression with Manual Gradient Calculation (3 pts)

We consider the logistic regression model

$$z = XW + b$$
$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where

- $X \in \mathbb{R}^{N \times d}$
- $W \in \mathbb{R}^{d \times 1}$
- $b \in \mathbb{R}$
- $\hat{y} \in \mathbb{R}^{N \times 1}$

Use flattened image vectors, so typically $d = 784$.

Requirements

1. Prepare the balanced binary dataset described above.
2. Flatten images into vectors.

3. Create two input versions:

- **raw-like scale**: values in $[0, 255]$
- **normalized scale**: values in $[0, 1]$

4. Initialize parameters: W, b randomly

5. Implement the forward pass manually:

- linear logits z ,
- sigmoid predictions $\hat{y}$.

6. Implement the **binary cross-entropy loss manually**:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

7. Compute the gradients **manually**, using formulas derived in class:

$$\begin{aligned}\frac{\partial L}{\partial z} &= \hat{y} - y \\ \frac{\partial L}{\partial W} &= \frac{1}{N} X^T (\hat{y} - y) \\ \frac{\partial L}{\partial b} &= \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)\end{aligned}$$

8. Implement manual gradient descent updates:

- update W and b yourself,
- do **not** use `.backward()` or `.grad`.

9. Train for at least **100 epochs**.

10. Track:

- training loss,
- validation loss,
- training accuracy,
- validation accuracy.

Plots

Plot:

- training loss vs epoch,
- validation loss vs epoch,
- training accuracy vs epoch,
- validation accuracy vs epoch.

Required Explanation (Markdown)

Explain:

- why logistic regression is suitable for binary classification,
 - why sigmoid is used,
 - why the derivative simplifies to $\frac{\partial L}{\partial z} = \hat{y} - y$,
 - why we can implement gradient descent here without autograd.
-

Task 2 — Logistic Regression: Manual Gradients vs Autograd (2 pts)

In this task you will verify your manual gradient calculations.

Requirements

Using the **same dataset** and **same model**, compare:

- manual gradient computation,
- autograd gradient computation.

Steps

1. Pick one mini-batch or a small subset of training data.
2. Compute:
 - manual $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$,
 - autograd versions using `requires_grad=True` and `loss.backward()`.
3. Compare the values numerically.

You may report:

$$\left\| \nabla_W^{\text{manual}} - \nabla_W^{\text{autograd}} \right\|$$

and similarly for b .

Required Explanation (Markdown)

Discuss:

- why the two gradients should match,
 - possible reasons for small numerical differences,
 - why checking gradients is useful when implementing models from scratch.
-

Task 3 — MLP with 1 Hidden Layer (3 pts)

Now move from logistic regression to a neural network with one hidden layer.

Use the model

$$\begin{aligned}a^{(0)} &= x \\z^{(1)} &= a^{(0)}(W^{(1)})^T + b^{(1)} \\a^{(1)} &= g(z^{(1)}) \\z^{(2)} &= a^{(1)}(W^{(2)})^T + b^{(2)} \\\hat{y} &= \sigma(z^{(2)})\end{aligned}$$

Choose a hidden activation such as `tanh` (see

en.wikipedia.org/wiki/Hyperbolic_functions)

Requirements

1. Implement all parameters manually as tensors:

- $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}$

2. Implement the forward pass manually.

3. Use manual binary cross-entropy.

4. Use autograd for gradients:

- call `loss.backward()`,
- update parameters manually inside `torch.no_grad()`.

5. Train for at least **100 epochs**.

6. Report training and validation:

- loss,
- accuracy.

Plots

Plot:

- training loss vs epoch,
- validation loss vs epoch,
- training accuracy vs epoch,
- validation accuracy vs epoch.

Required Explanation (Markdown)

Explain:

- why an MLP can model more complex decision boundaries than logistic regression,
- the role of the hidden activation function,

- why we still need a sigmoid in the output layer.
-

Task 4 — MLP with 2+ Hidden Layers (2 pts)

Extend the previous model to **two (or more) hidden layers**.

Example architecture:

$$\begin{aligned}a^{(0)} &= x \\z^{(1)} &= a^{(0)}(W^{(1)})^T + b^{(1)}, & a^{(1)} &= g(z^{(1)}) \\z^{(2)} &= a^{(1)}(W^{(2)})^T + b^{(2)}, & a^{(2)} &= g(z^{(2)}) \\z^{(3)} &= a^{(2)}(W^{(3)})^T + b^{(3)}, & \hat{y} &= \sigma(z^{(3)})\end{aligned}$$

Requirements

1. Implement a 2-hidden-layer MLP manually.
2. Train it on the same binary dataset.
3. Compare its performance with:
 - logistic regression,
 - 1-hidden-layer MLP.

Required Comparison

Report for all three models:

- final train loss,
- final validation loss,
- final train accuracy,
- final validation accuracy.

Required Explanation (Markdown)

Discuss:

- whether deeper models improved performance,
- whether the extra hidden layer was helpful,
- why a deeper model may help or may fail to help on this binary task.